IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Bases de datos NoSQL

Cuándo conviene guardar los datos como documentos

Sprint 3 · S03_02 · presentación (15 min)

LA IDEA · DEL JOIN AL DOCUMENTO

El mismo préstamo, guardado de otra forma

SQL: TRES TABLAS Y DOS JOIN

```
SELECT p.id_prestamo, p.dias, p.tarde,
       l.id_libro, l.titulo, s.id_socio, s.nombre
FROM prestamos p
JOIN libros l ON l.id_libro = p.id_libro
JOIN socios s ON s.id_socio = p.id_socio
WHERE p.id_prestamo = 2;
```

NOSQL: UN DOCUMENTO

```
{"_id": 2,
 "dias": 24,
 "tarde": True,
 "libro": {"id": 1, "titulo": "El faro del norte"},
 "socio": {"id": 2, "nombre": "Beto"}}
```

Embebido

El libro y el socio van **dentro** del préstamo, como **subdocumentos**: se lee todo de una vez, **sin JOIN**.

Un diccionario

Un **diccionario** de Python es una lista de pares **nombre: valor** entre llaves. Un valor puede ser **otro diccionario**.

El coste

El título del libro se escribe **en cada préstamo**. Lo veremos en la diapositiva 5.

LA IDEA · EL VOCABULARIO

Colecciones y documentos

En SQL	En MongoDB
Tabla	**Colección**: sin esquema fijo
Fila	**Documento**, parecido a un diccionario
Columna	**Campo**
Clave primaria	`_id`
Base de datos: conjunto de tablas	Base de datos: conjunto de **colecciones**

LIBRO IMPRESO

```
{"titulo": "El faro del norte",
 "paginas": 320}
```

AUDIOLIBRO

```
{"titulo": "Radio Teide",
 "duracion_min": 410,
 "narrador": "Lola"}
```

Sin esquema fijo

Dos documentos de la **misma colección** pueden traer **campos distintos**. En SQL habría que cambiar la tabla (**ALTER TABLE**).

LA IDEA · SQL O NOSQL

No es «uno u otro»

	Relacional (SQL)	Documental (NoSQL)
Estructura	Tablas fijas, con relaciones	Documentos flexibles
Mejor para	Préstamos y multas: cifras que **deben cuadrar**	Reseñas, búsquedas del catálogo, fichas con **atributos distintos**
Cambiar la estructura	Hay que migrar	Basta añadir un campo
Punto débil	Rigidez	**Duplicación**

Muchos proyectos usan las dos. La pregunta es: ¿qué tipo de dato tengo y qué voy a hacer con él?

LA IDEA · EL PUNTO DÉBIL

Si el libro cambia de título...

SQL: 1 fila cambiada, en libros

id_libro	titulo
1	El faro del norte, nueva edición

Los préstamos solo guardan **id_libro**: no hay nada más que tocar.

MongoDB: 3 documentos cambiados

_id	libro.titulo
1	El faro del norte, nueva edición
2	El faro del norte, nueva edición
8	El faro del norte, nueva edición

```sql
UPDATE libros
SET titulo = 'El faro del norte, nueva edición'
WHERE id_libro = 1;
```

```python
nuevo = "El faro del norte, nueva edición"
prestamos.update_many(
    {"libro.id": 1},
    {"$set": {"libro.titulo": nuevo}})
```

Con documentos, el título está copiado en cada préstamo: si cambia, hay que cambiarlo en 3 sitios.

LA IDEA · CONSULTAR CON FIND

La misma pregunta que en SQL

SQL (tras el **JOIN**)		MongoDB: **find(filtro)**	Salen
`WHERE s.barrio = 'Centro'`	→	`{"socio.barrio": "Centro"}`	**6 documentos**
`WHERE dias > 20`	→	`{"dias": {"$gt": 20}}`	**3 documentos**
`WHERE l.genero IN ('ciencia', 'novela')`	→	`{"libro.genero": {"$in": ["ciencia", "novela"]}}`	**8 documentos**

Un filtro es un diccionario

find(filtro) devuelve los documentos que lo cumplen. Para tenerlos en una lista: **list(...)**.

Los $ son operadores

$gt es «greater than», mayor que; **$in** es «dentro de» una lista.

El punto entra en el subdocumento

"socio.barrio" es el campo **barrio** **dentro** de **socio**.

LA IDEA · AGREGACIONES

aggregate: el GROUP BY de MongoDB

```
pipeline = [
    {"$match": {"dias": {"$gt": 8}}},
    {"$group": {"_id": "$socio.nombre",
                "prestamos": {"$sum": 1},
                "dias_medios": {"$avg": "$dias"}}},
    {"$sort": {"dias_medios": -1}},
]
```

Resultado de la pipeline

_id	prestamos	dias_medios
Beto	1	24,0
Carla	3	18,3
Ana	3	17,0
Dani	1	12,0

colección
10 documentos

→

$match ≈ WHERE
8 documentos

→

$group ≈ GROUP BY
4 grupos, uno por socio

→

$sort ≈ ORDER BY
de más a menos días medios

LO QUE VIENE · EL CUADERNO DE NOSQL

Practicar sin instalar nada

ASÍ EMPIEZA EL CUADERNO

```
import mongomock
cliente = mongomock.MongoClient()
reservas = cliente["turisdata"]["reservas"]
reservas.insert_many(documentos)
reservas.count_documents({})
```

mongomock

Es MongoDB «de mentira», **en memoria**: no hay que instalar nada. El código de **pymongo** es el mismo que con un servidor real.

En parejas

Una sola pantalla; **cambiáis de rol** en cada ejercicio.

Celda a celda

Mayús+Intro ejecuta y pasa a la siguiente. **No** uséis «Ejecutar todo».

Hasta el ejercicio 5

En clase llegáis hasta el ejercicio 5. Lo demás es para **casa**.

EL RETO · PRIMER MODELO EN PRODUCCIÓN

Qué parte del reto prepara esto

Ningún paso del reto

Esta presentación es **refuerzo de datos**: no prepara ninguno de los seis pasos del reto.

Lo usaréis después

En el Sprint 5, el CRUD se hace también con **mongomock**, además de **sqlite3**.

El día 1 del sprint: 200 minutos

10 min	40 min	105 min	15 min	30 min
Planning	Presentación SQL	Scripts SQL 01–06 en DB Browser	**Esta**	Cuaderno NoSQL

ANTES DE ABRIR EL CUADERNO

Lo que hay que llevarse

Documentos

La información relacionada va **dentro** de un documento: se lee de una vez, **sin JOIN**.

No es uno u otro

SQL para cifras que deben cuadrar; **documentos** para datos flexibles. Su punto débil: la **duplicación**.

Mismas preguntas

find ≈ **WHERE** y **aggregate** con **$group** ≈ **GROUP BY**: **otro idioma**, la misma idea.

Ahora: cuaderno de NoSQL en parejas, 30 minutos, hasta el ejercicio 5.